Step 2 — The RAG Layer and the Skill Extractor — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — the resume store and the retriever
- Hint 2 — one distance per skill, and the extractor

Hint 1 — the resume store and the retriever

This is Doc08's RAG, over the resume:

- `src/rag/resume_store.py` → `build_resume_store(resume_text)`: split with `RecursiveCharacterTextSplitter(chunk_size=300, chunk_overlap=40).split_text(...)`, embed with `OpenAIEmbeddings(model="text-embedding-3-small")`, and `collection.add(ids, documents=chunks, embeddings=...)` to a fresh Chroma collection (delete-then-create so re-runs are clean).
- `src/rag/retriever.py` → `best_distance(collection, skill)`: `collection.query(query_embeddings=[embed_query(skill)], n_results=1)` and return `results["distances"][0][0]` — or a big number (2.0) if the collection is empty.

Hint 2 — one distance per skill, and the extractor

`best_distance` returns exactly one number per skill — which is all Step 1's `match_skills` needs. Lower = the resume covers it; higher = it doesn't. That one number is the bridge between RAG and the decision engine.

The extractor is one structured chain:

```
extract_required_skills(model, job_text):
    prompt = "list the required skills/technologies as short names ..."
    chain = prompt | model.with_structured_output(RequiredSkills)
    return chain.invoke({"job": job_text})
```

Keep the skill names short ("Python", not "5 years of Python") so they make clean search queries. The live run is Step 3. The whole file set is in the Solution.